最实用的Makefile教程 真的很简单（搞不明白网上的教程写那么复杂干嘛）-CSDN博客

本文从基础出发，详细讲解了Makefile的显式规则、变量、隐含规则及通配符的使用，通过实例演示如何编写高效简洁的Makefile文件，帮助读者快速掌握Makefile的编写技巧。

摘要生成于 C知道 ，由 DeepSeek-R1 满血版支持， 前往体验 >

前言

其实我的要求不高，我就是想要写个 Makefile ，把我那些需要反复编译的文件处理一下就可以了，所以我当时就拼命地在网上找关于Makefile的教程，结果看到的教程都是啰里啰嗦一大堆，看得我云里雾里。

大家要知道，在一些大公司里面，有一个职业是专门写Makefile的，可是我就想用Makefile完成那么一点基础的工作，你给我整那么多乱七八糟的东西干嘛。

在经受网上各种教程的毒打之后，我实在是忍不住了，我自己写一个教程算了。大家看这篇文章的时候，如果你也和我一样，一开始只是想解决最基础的问题，你就只要看第一个章节就可以了，后面的大家视情况而定。

一、Makefile的显式规则

首先，在Makefile中，#代表着注释，这个是不会被编译进去的。

其次，Makefile的基本语法是：
目标文件：依赖文件
[TAB]指令

大家注意，在Makefile里，有一个很反人类的规定，指令前必须打一个[Tab]键，按四下空格会报错。

越是接近目标文件的命令，就越是要写在前面。因为程序是按照递归的方式进行依赖文件查找的，看到第一行有一个没见过的依赖文件，就往下一行进行查找，以此类推。

但是有些同学反映，不按照这个顺序来写好像也不会报错，我觉得这可能是和版本有关系，不过保险起见，我建议大家还是按照规范来写Makefile。

举个例子：

```
hello:hello.o
    gcc hello.o -o hello

hello.o:hello.S
    gcc -c hello.S -o hello.o

hello.S:hello.i
    gcc -S hello.i -o hello.S

hello.i:hello.c
    gcc -E hello.c -o hello.i
cpp1234567891011
```

假设当前文件夹中有hello.c和Makefile两个文件，当我们在终端输入make指令的时候，就会自动编译出hello.o，hello.S，hello.i以及hello可执行文件。

可是，我们又不想要这些不相关的文件，想对这些文件做一些操作，我们把这样的操作叫做伪目标，标志位.PHONY:

在上述代码的最后面加上：

```
.PHONY:
clear:
    rm hello.o hello.S hello.i
cpp123
```

.PHONY:这是固定格式，不能变的，但是下面的clear是自己取的名字，你随便取什么名字都可以，但是clear比较直观。

这样，当我们执行make clear指令后，将只剩下hello.c 和 hello可执行文件。

再来个复杂一点的例子：

```
# 目标文件：test
# 现有文件：program1.c program1.h program2.c program2.h main.c main.h

test:program1.o program2.o main.o
    gcc program1.o program2.o main.o -o test

program1.o:program1.c
    gcc -c program1.c -o program1.o

program2.o:program2.c
    gcc -c program2.c -o program2.o

main.o:main.c
    gcc -c main.c -o main.o

.PHONY:
clean_all:
    rm program1.o program2.o main.o
cpp12345678910111213141516171 8
```

到这里，Makefile就学会了，就可以用来做事情了，就是这么简单。

后面的内容无所谓你看不看了，放学了。。。

二、变量

符号	含义
=	替换
+=	追加
:=	恒等于

如果我们写TAR = test，就表示下面的代码中，我们可以用TAR代表test文件。
如果再写TAR += test1，就表示TAR代表test和test1。

CC:= gcc 就表示下面写gcc的地方全部可以用CC代替，因为gcc这个是不会变的，是常量，所以可以用恒等于替换，这个不能用+=。

当我们要调用这些变量的时候，就直接使用$(变量)的方式进行调用。

举个例子，对比上面那一段代码，可以修改成下面的样子：

```
TAR = test
OBJ = program1.o program2.o main.o
CC := gcc

$(TAR):$(OBJ)
    $(CC) $(OBJ) -o $(TAR)

program1.o:program1.c
    $(CC) -c program1.c -o program1.o

program2.o:program2.c
    $(CC) -c program2.c -o program2.o

main.o:main.c
    $(CC) -c main.c -o mian.o

.PHONY:
clean_all:
    rm $(OBJ)
powershell12345678910111213141516171819
```

三、隐含规则

符号	含义
%.o	任意的.o文件
*.o	所有的.o文件

于是，我们又可以把上面的代码简化一下：

```
TAR = test
OBJ = program1.o program2.o main.o
CC := gcc

$(TAR):$(OBJ)
    $(CC) $(OBJ) -o $(TAR)

%.o:%.c
```

```
    $(CC) -c %.c -o %.o

.PHONY:
clean_all:
    rm $(OBJ)
powershell12345678910111213
```

四、通配符

符号	含义
$^	所有依赖文件
$@	所有目标文件
$<	所有依赖文件的第一个文件

所以，上诉代码还可以简化成下面这个样子：

```
TAR = test
OBJ = program1.o cube.o main.o
CC := gcc
RMRF := rm

$(TAR):$(OBJ)
    $(CC) $^ -o $@
%.o:%.c
    $(CC) -c $^ -o $@

.PHONY:
cleanall:
    $(RMRF) $(OBJ)
powershell12345678910111213
```

嗯，基本上我一下子就想到这么多了。

内容来源：csdn.net

作者昵称：Nire_谒羽

原文链接：https://blog.csdn.net/Nire_Yeyu/article/details/106373974

作者主页：https://blog.csdn.net/Nire_Yeyu

实付 元

使用余额支付

点击重新获取

扫码支付

钱包余额 0

抵扣说明：

1.余额是钱包充值的虚拟货币，按照1:1的比例进行支付金额的抵扣。
2.余额无法直接购买下载，可以购买VIP、付费专栏及课程。

余额充值

举报

AI 搜索 智能体 AI 编程 AI 作业助手

隐藏侧栏

程序员都在用的中文IT技术交流社区

专业的中文 IT 技术社区，与千万技术人共成长

关注【CSDN】视频号，行业资讯、技术分享精彩不断，直播好礼送不停！

客服 返回顶部